

BigBird v9 处理器设计说明

本文面向“宏观 + 微观”两个层次：先给出整机流水线与数据/控制通路全貌，再逐个模块列出**输入、输出与作用**，并配合典型指令走查。

v9 在 v8（RV32I 教学流水线）基础上完成四项升级：

RV32IM 乘法、I/D-Cache、机器模式中断、动态分支预测。

0. 版本定位与文件清单

能力	v8	v9
指令集	RV32I	RV32IM （新增 8 条乘法）
取指	直连 IROM	I-Cache （直接映射）
访存	直连 DRAM	D-Cache （写直达、写不分配）
控制流	静态“不跳转”	2 位 BHT + BTB 动态预测
异常	ecall	ecall + 机器模式软件/定时器中断
CSR	mepc/mcause/mstatus/mtvec	扩展 mie/mip/mscratch/mtime/mtimecmp
可观测性	写回口	写回口 + Cache/预测器命中计数（波形可见）

v9 目录文件：

文件	角色
myCPU.sv	顶层：把各模块连成流水线
IFU.sv	取指级（PC 维护、重定向/预测/暂停）
ICache.sv	指令缓存
Branch_Predictor.sv	动态分支预测器（BHT+BTB）
IDU.sv	译码级（两级寄存 + 译码 + CSR 读口）
Reg_Stack.sv	通用寄存器堆 + CSR 空间封装
RegisterFile.sv	32×32 通用寄存器堆
CSR.sv	机器模式 CSR 与中断控制
EXU.sv	执行级（ALU/MDU、分支判定）
ALU.sv	整数算术/逻辑/移位/比较
MDU.sv	RV32M 乘法/除法单元
LSU.sv	访存级（多级流水 + 多级前递观察点）

文件	角色
DCache.sv	数据缓存
WBU.sv	写回级（最终提交值选择）
Data_hazard.sv	前递仲裁（只产生选择编码）
Control.sv	前递多路选择、暂停、冲刷、重定向、中断接收
sext.sv / add.sv / Reg.sv	通用小模块（符号扩展、共享加法器、通用寄存器模板）
para.sv	全局参数与编码宏

1. 整机结构：功能 5 级、物理 8 级

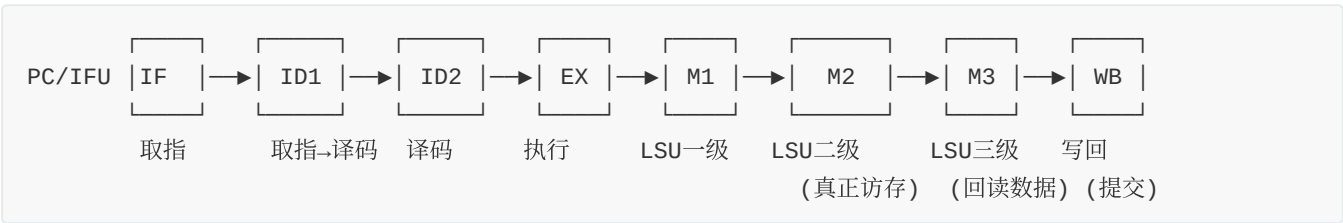
先给结论，避免“几级流水线”的混淆：

- 按功能级划分，它仍是一颗经典 5 级流水线：IF / ID / EX / MEM / WB，对应模块 IFU / IDU / EXU / LSU / WBU。这也是项目 README 的描述（v6“引入 IF/ID/EX/MEM/WB 阶段划分”）。
- 按寄存器边界看，物理上更深：IDU 把译码拆成 2 拍（inst、inst_d），LSU 把访存拆成 3 拍（reg、reg_pipe、reg2），加上 IF/EX/WB 各 1 拍，共 8 个流水寄存器，一条指令从取指到写回约 6~7 拍；稳态吞吐仍是 1 IPC。
- 多出来的拍不是冗余，而正好对应 Data_hazard 里的 MEM / MEM_PIPE / MEM2 三个前递观察点，也方便教学时观察访存时序。

实测（v9，复位释放后）：第一条指令在第 21 个 tick 取指、第 27 个 tick 在写回级可见，即取指→写回约 6~7 拍；两条长度相差 20 的直线程序，周期数也只相差 20 —— 即 1 IPC。

所以：“5 级”是功能级视角，“8 级”是物理寄存器视角，两者说的是同一颗 CPU。

下面按物理寄存器边界展开：



对应到代码里的寄存器（物理视角）：

流水级	承载寄存器	说明
IF	IFU.pc	当前取指地址；inst 由 I-Cache 组合给出
ID1	IDU.inst/snpc/pc/valid	从 IFU 接住原始指令
ID2	IDU.inst_d/.../pt_d	稳定一拍后组合译码（产生全部控制信号）
EX	EXU.*_reg	ALU/MDU 运算、分支条件、跳转目标

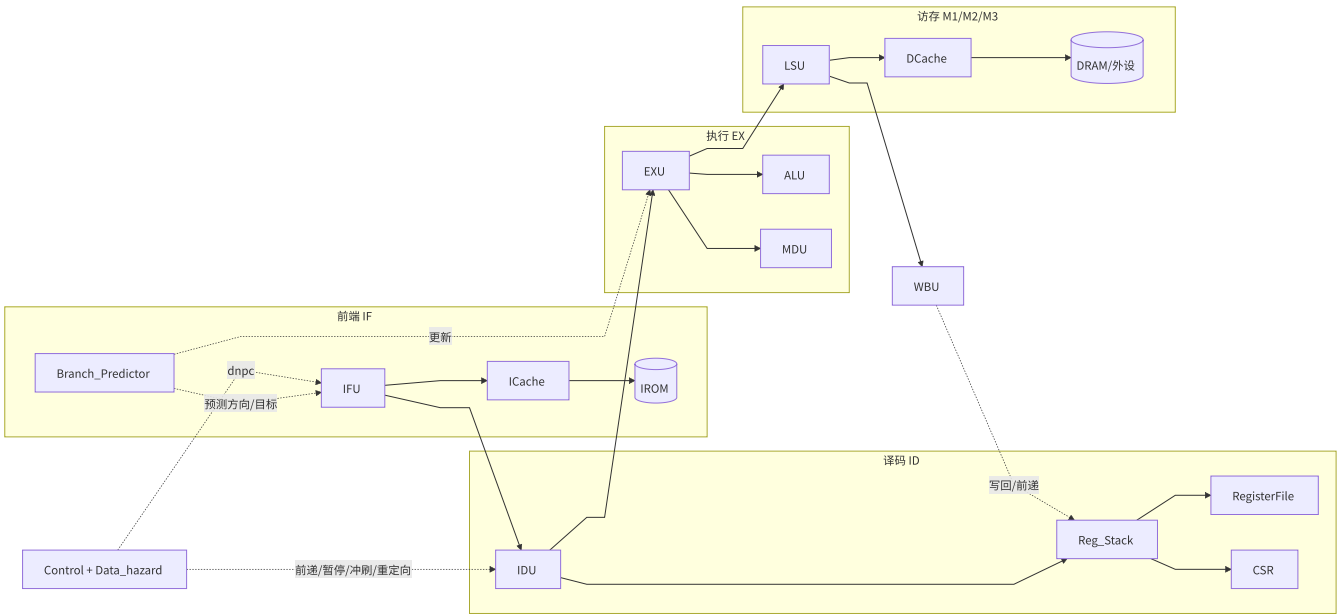
流水级	承载寄存器	说明
M1	LSU.*_reg	EX 结果进入访存级第一级
M2	LSU.*_reg_pipe	真正驱动外部总线 (addr/wen/wdata/mask)
M3	LSU.*_reg2	锁存读回数据, 按 funct3 做 load 扩展
WB	WBU.*_reg	最终选择写回值并提交

多级前递：因为流水线深，Data_hazard 会同时在 EX/M1/M2/M3/WB 五个点比较目标寄存器号，Control 负责把对应值选出来送回译码级。这解决了“结果还很远，消费者却已到译码级”的时间差。

功能级 ↔ 物理寄存器的对应关系：

功能级（5 级）	物理寄存器（8 个）	备注
IF	IFU.pc	取指
ID	IDU.inst → IDU.inst_d	译码在 inst_d 上组合完成
EX	EXU.*_reg	ALU/MDU/分支判定
MEM	LSU.*_reg → .*_reg_pipe → .*_reg2	三级访存流水
WB	WBU.*_reg	提交

1.1 数据通路全貌（Mermaid）



2. 顶层 myCPU

2.1 对外接口

方向	信号	位宽	作用
in	cpu_clk, cpu_rst	1	时钟、高有效同步复位
out	irom_addr	32	指令地址（由 I-Cache 转发给 IROM）
in	irom_data	32	IROM 指令数据（组合读）
out	perip_addr/wen/mask/wdata	—	数据总线（由 D-Cache 转发给 DRAM/外设）
in	perip_rdata	32	数据总线读回
out	debug_wb_have_inst/pc/ena/reg/value	—	差分测试写回探针

2.2 顶层信号分组（便于阅读）

- 取指组：IFU_pc/IFU_snpc/IFU_inst/IFU_valid，I-Cache 相关 icache_*。
- 译码组：IDU_*（控制信号、add1/add2、CSR/系统标志、前递后的 EXU_rs1_in/rs2_in）。
- 执行组：EXU_*（EX_result、EXU_branch_flag/jump_flag、EXU_Ex_result）。
- 访存组：LSU_*（LSU_Rdata、LSU_Ex_result、多级 *_pipe/*_wb）。
- 写回组：WBU_*（WBU_rd_value 等）。
- 控制组：dnpc/dnpc_flag、IFU_stall、EXU_inst_clear、bp_*、intr_take。

顶层不做算术，只做“连线 + 命名”，真正的逻辑都在子模块里。阅读顺序建议：
IFU → IDU → EXU → LSU → WBU，再回头看 Control/Data_hazard 如何把它们粘起来。

3. 取指前端

3.1 IFU（取指级）

输入	作用
clock/reset	时钟/复位
dnpc[31:0], dnpc_flag	执行级确认的 真实重定向 目标（最高优先级）
pred_taken, pred_target[31:0]	分支预测给出的 投机跳转
irom_data[31:0]	来自 I-Cache 的指令
stall	load-use/中断等导致的取指暂停
ready	后级是否可接收（来自 IDU）

输出	作用
pc[31:0]	当前取指地址
snpc[31:0]	pc+4（顺序下一拍）
inst[31:0]	取到的指令（= irom_data）
valid	恒为 1；是否被真正接收由 ready/stall 决定

PC 更新优先级（IFU.sv）：

```
reset > stall(保持) > dnpc_flag(真实重定向) > pred_taken(预测跳转) > pc+4
```

作用：把“顺序取指、暂停、预测、纠正”四种前端行为收敛到一个 always_ff 里。

它在做什么：维护唯一的 PC 状态。每个时钟沿只做一件事——决定下一个 PC：
被暂停就保持、有真实重定向就跳、有预测就投机跳、否则顺序 +4。指令数据本身不由它加工，而是把 I-Cache 的输出原样送出。

关键代码（IFU.sv）

```
assign valid = 1'b1;
assign snpc  = pc + 4;
assign inst  = irom_data;

always_ff @(posedge clock) begin
    if (reset)                pc <= 32'h0;
    else if (stall & valid & ready) pc <= pc;           // 暂停：保持
    else if (dnpc_flag & valid & ready) pc <= dnpc;      // 真实重定向
    else if (pred_taken & valid & ready) pc <= pred_target; // 预测跳转
    else if (valid & ready)      pc <= snpc;           // 顺序 +4
end
```

与后级的联系：pc/snpc/inst/valid 直接进入 IDU 的第一级寄存器
(IDU.inst <= inst_in)；同一拍的 pred_taken/pred_target 也一并送入 IDU，
随指令一路下传到 EXU，最终由 Control 用来判断预测是否正确。

3.2 ICache（指令缓存，直接映射）

输入	作用
clk/reset	时钟/复位
cpu_addr[31:0]	IFU 的取指地址
mem_rdata[31:0]	来自 IROM 的指令

输出	作用
cpu_rdata[31:0]	命中取缓存、缺失旁路 IROM

输出	作用
mem_addr[31:0]	转发给 IROM 的地址
hit/miss	本次访问结果
hit_cnt/miss_cnt[31:0]	累计命中/缺失计数（波形可观察命中率）

组织：行 = 1 个字；index = addr[INDEX+1:2]，tag = addr[31:INDEX+2]，另配 valid 位。命中判定 valid[index] && tag==tag_in。

缺失处理：本验证平台 IROM 为零延迟组合读，因此缺失时直接旁路并在本拍末尾回填缓存行，不插入 stall（若换成多周期存储器，只需把 miss 接到 IFU.stall）。

它在做什么：在 IFU 和 IROM 之间做一层“查表”——命中就用缓存里的字，缺失就用 IROM 组合给出的字，并把这一行填进缓存。它不改变取指语义，因此对上层的 IFU 完全透明。

关键代码（ICache.sv）

```
assign index  = cpu_addr[INDEX_BITS+1:2];
assign tag_in = cpu_addr[31:INDEX_BITS+2];
assign hit    = valid[index] && (tag[index] == tag_in);
assign cpu_rdata = hit ? data[index] : mem_rdata;    // 命中取缓存，缺失旁路
assign mem_addr  = cpu_addr;

always_ff @(posedge clk)
    if (!reset && miss) begin                                // 缺失回填
        valid[index] <= 1'b1; tag[index] <= tag_in; data[index] <= mem_rdata;
    end
```

与后级的联系：cpu_rdata 就是 IFU.inst，随 pc/snpc 一起送入 IDU。
hit/miss 目前只用于计数与波形；若把 miss 接到 IFU.stall，即可扩展为带缺失惩罚的缓存（见 §11）。

3.3 Branch_Predictor（动态分支预测）

输入	作用
fetch_pc[31:0]	IF 级查询地址
update_en	EX 级是否有控制流指令需要更新
update_pc[31:0]	被更新指令的 PC
update_taken	实际是否跳转
update_target[31:0]	实际跳转目标
update_correct	本次预测是否正确（统计用）

输出	作用
<code>pred_taken</code>	是否预测跳转
<code>pred_target[31:0]</code>	预测目标
<code>pred_cnt/hit_cnt/miss_cnt</code>	预测次数/正确/错误

结构：

- **BHT**：64 项 2 位饱和计数器，`>=2` 预测 taken；
- **BTB**：64 项 `{valid, tag, target}`，保存上次跳转目标。

规则：`BTB 命中 且 BHT 最高位=1` → 预测 taken。

更新：taken 时计数器 +1，否则 -1；taken 时刷新 BTB 目标。

校验：在 EX 级把预测与真实结果比较，不一致则 `Control` 发出 `dnpc` 冲刷前端。

它在做什么：猜“这一条控制流指令会不会跳、跳去哪”。IF 级用地址查表给出预测，EX 级拿到真实结果后修正表项。它只影响“提前走哪条路”，不影响最终语义——猜错时由 `Control` 把错误路径冲刷掉。

关键代码（`Branch_Predictor.sv`）

```
// 预测
assign btb_hit      = valid[fidx] && (tag[fidx] == ftag);
assign pred_taken   = btb_hit && bht[fidx][1];    // BTB 命中且计数器最高位为 1
assign pred_target  = target[fidx];

// 更新 (EX 级)
if (update_en) begin
    if (update_taken) begin if (bht[uidx] != 2'b11) bht[uidx] <= bht[uidx]+1; end
    else                begin if (bht[uidx] != 2'b00) bht[uidx] <= bht[uidx]-1; end
    if (update_taken) begin valid[uidx] <= 1; tag[uidx] <= utag; target[uidx] <= update_target;
end
end
```

与后级的联系：`pred_taken/pred_target` 一方面给 `IFU` 决定下一 PC，另一方面随指令进入 `IDU→EXU`；到 EX 后 `Control` 用它和真实结果比较（`bp_mispredict`），再把 `bp_update_en/taken/target` 写回本模块。这也是本文档里唯一一个“反向更新”的模块：信息在前端预测，在执行级回灌。

4. 译码级 IDU 与寄存器/CSR

4.1 IDU（译码级）

IDU 内部含两级流水寄存：`inst`（ID1）与 `inst_d`（ID2）。组合译码全部基于 `inst_d`，因此控制信号稳定、时序压力小。

主要输入

输入	作用
inst_in/snpc_in/pc_in	来自 IFU 的指令、PC+4、PC
flush(=dnpc_flag)	重定向时把本级灌成 NOP
stall	暂停：本级不推进
rd_value/csrd/rd/R_wen/csr_wen	来自 WB 的写回与 CSR 写使能
EXU_rs1_in/EXU_rs2_in	经 Control 前递后的真实操作数
pred_taken_in/pred_target_in	IF 级每个 PC 的预测（随指令下传）
intr_take	本拍接收中断（触发 CSR 陷入）
valid_last/ready_next	与前后级的 valid/ready 握手

主要输出

输出	作用
rd_next/funct3/rs1/rs2	寄存器读写索引
alu_opcode[4:0]	ALU/MDU 运算编码（最高位区分 M 扩展）
add1_value/add2_value	统一后的 ALU 双操作数
mem_wen/mem_ren/inv_flag/branch_flag/jump_flag	访存与控制流标志
branch_pc	分支目标（pc_d + imm）
csr_wen_next[15:0]	各 CSR 写使能位图
mret_flag/ecall_flag/fence_i_flag	系统指令识别
mepc_out/mtvec_out/intr_pending	来自 CSR 的陷入入口与中断待处理
pred_taken/pred_target	随指令送到 EX 的预测信息
pc_out	被译码指令的 PC（= pc_d，用于陷入 mepc）

译码要点

- 1. 指令族识别：由 7 位 opcode 得到 is_R/is_I0/is_I1/is_I2/is_S/is_B/is_U0/is_U1/is_J/is_SYS。
- 2. RV32M：is_MEXT = is_R && funct7==7'b0000001，再按 funct3 选 mul/mulh/mulhsu/mulhu/div/divu/rem/remu。
- 3. 统一操作数：add_src1/add_src2 把不同格式指令映射成 ALU 的双输入，执行级无需再关心指令格式。
- 4. 前递写回值：rd_value_next 对跳转返回 snpc、对系统指令返回读出的 CSR 值。

它在做什么：把 32 位机器码翻译成“一整包控制信号 + 操作数”。它是全机最“杂”的一级，但输出全部是组合信号，真正打拍的是两级 inst~inst_d 寄存。

关键代码（IDU.sv）

```
// ① 两级流水寄存
always_ff: inst    <= inst_in;    // ID1: 接住取指结果
always_ff: inst_d <= inst;        // ID2: 稳定一拍后组合译码

// ② 指令族识别 + RV32M
assign is_R      = (opcode == `R_opcode);
assign is_MEXT = is_R && (oprand == `funct7_m);
assign m_alu_opcode = (funct3==3'b000) ? `alu_mul : ... : `alu_remu;
assign alu_opcode   = is_MEXT ? m_alu_opcode : cond_add ? `alu_add : ...;

// ③ 把不同指令格式统一成 ALU 双操作数
assign add_src1 = is_U0 ? 32'd0 : (is_J || is_U1) ? pc_d : EXU_rs1_in;
assign add_src2 = (is_R || is_B) ? EXU_rs2_in : ... : imm;

// ④ 无需加工即可写回的值
assign rd_value_next = jump_flag ? snpc_d : is_SYS ? csrs : 0;
```

与后级的联系：IDU 的输出几乎“定义”了后面所有模块的行为：

- add1/add2/alu_opcode 决定 EXU 算什么；mem_wen/mem_ren 决定 LSU 是否访存；
- branch_flag/jump_flag/branch_pc 决定 Control 是否重定向；
- csr_wen_next/rd_value_next 则一路透传到 WBU 与 CSR；
- pred_taken/pred_target 继续下传供 Control 校验。

4.2 Reg_Stack + RegisterFile + CSR

Reg_Stack：把通用寄存器堆与 CSR 空间封成一个读入口。

输入	作用
rs1/rs2/rd/rd_value/R_wen	通用寄存器读写
csr_addr[31:0]	CSR 读地址（来自 imm）
csr_wen[15:0], csrd[31:0]	CSR 写使能与写数据
ecall_flag/mret_flag/intr_take	陷入/返回事件
pc[31:0]	被陷入指令 PC

输出	作用
rs1_value/rs2_value/a0_value	通用寄存器读值（a0 供调试）
csrs[31:0]	按 csr_addr 译码后的 CSR 读值
mepc_out/mtvec_out	陷入/返回入口
intr_pending	mstatus.MIE & (mie&mip)

RegisterFile：双读口单写口，x0 恒 0（写口被钳位）。

CSR：机器模式 CSR 集合与中断判决。

输入	作用
pc/ecall_flag/mret_flag/intr_take	陷入/返回事件
csrd[31:0], csr_wen[15:0]	软件写
intr_cause（内部产生）	中断原因

输出	作用
mstatus/mie/mip/mtvec/mepc/mcause/mscratch	机器寄存器
mtime/mtimecmp	教学用内部定时器
intr_pending	待处理且全局使能

中断判决：pending = mie & mip，优先级 MEI(11) > MSI(0) > MTI(7)。

陷入：mepc<=pc、mcause<=0x8000_0000|cause、MPIE<=MIE、MIE<=0、入口 mtvec。

返回：MIE<=MPIE、MPIE<=1、PC<=mepc（PC 重定向由 Control 完成）。

它在做什么：保存 32 个通用寄存器与一整套机器模式 CSR，对外只暴露一个“读口”和一个“写口”。读口在译码级组合给出 rs1_value/rs2_value/csrs；写口在写回级由 WBU 驱动。中断/异常的进入与 mret 返回也在这里改状态。

关键代码（Reg_Stack.sv / RegisterFile.sv / CSR.sv）

```
// 读 CSR: 按地址把各机器寄存器并到一个返回口
assign csrs = (csr_addr==`CSR_mepc) ? mepc_out :
              (csr_addr==`CSR_mie) ? mie_out : ... : 32'd0;

// 通用寄存器: x0 钳位为 0; 组合读
assign wdata = (rd==5'd0) ? 32'd0 : rd_value;
assign rs1_value = rf[rs1_addr];

// CSR 陷入/返回状态机
if (intr_take || ecall_flag) begin // 陷入
    mepc <= pc; mcause <= {1'b1,27'b0,intr_cause};
    mstatus.MPIE <= mstatus.MIE; mstatus.MIE <= 0;
end else if (mret_flag) begin // 返回
    mstatus.MIE <= mstatus.MPIE; mstatus.MPIE <= 1;
end
```

与后级的联系：csrs 作为 rd_value_next 的一部分参与写回（CSR 读）；mepc_out/mtvec_out 送到 Control，分别用于 mret 返回与中断/异常入口；intr_pending 也送到 Control 参与中断接收判断。反向地，CSR 的软件写数据 csrd 来自 WBU（= EXU 算出的结果），寄存器写数据 rd_value 来自 WBU。

4.3 立即数生成（IDU 内）

立即数	格式	说明
imm_I	{{20{inst[31]}}, inst[31:20]}	addi/load/jalr/CSR
imm_S	{{20{inst[31]}}, inst[31:25], inst[11:7]}	store
imm_B	由 imm_S 重排 <<1	分支
imm_U	{inst[31:12], 12'd0}	lui/auipc
imm_J	位重排 <<1	jal

关键代码（IDU.sv）

```
assign imm_I = {{20{inst_d[31]}}, inst_d[31:20]};
assign imm_S = {{20{inst_d[31]}}, inst_d[31:25], inst_d[11:7]};
assign imm_B = {{19{inst_d[31]}}, inst_d[31], inst_d[7],
                inst_d[30:25], inst_d[11:8], 1'b0};
assign imm_U = {inst_d[31:12], 12'd0};
assign imm_J = {{11{inst_d[31]}}, inst_d[31], inst_d[19:12],
                inst_d[20], inst_d[30:21], 1'b0};
```

与后级的联系：这些 imm 最终进入 add_src2（ALU 第二操作数）或
branch_pc = pc_d + imm（分支目标），是 EXU 的两个关键输入来源。

5. 执行级 EXU

输入	作用
add1/add2[31:0]	ALU/MDU 双操作数
alu_opcode[4:0]	运算选择
inv_flag	分支条件取反（统一比较方向）
branch_flag/jump_flag/fetch_i_flag	控制流/系统标志
branch_pc[31:0]	分支目标
rd_value[31:0]	CSR/跳转类预置写回值
pred_taken_in/pred_target_in	随指令下传的预测信息
EXU_inst_clr	本拍需清空（重定向/停顿插泡）

输出	作用
EX_result[31:0]	运算/比较结果（比较结果在 bit0）

输出	作用
branch_pc_next/rd_value_next/...	打拍后送给 LSU 的控制与数据
pred_taken_next/pred_target_next	预测信息继续下传（供 Control 校验）
pc_out[31:0]	本条指令 PC

关键约定：分支条件被归一化到 EX_result[0]（必要时异或 inv_flag），因此 Control 只需读最低位即可判断是否跳转。

它在做什么：把译码级给来的两个操作数真正“算出来”，并把控制/预测信息再打一拍交给访存级。分支条件、跳转目标、CSR/ALU 结果都在这里产生。

关键代码（EXU.sv）

```
// ALU 与 MDU 并行，最高位选结果
assign exec_res  = alu_opcode_reg[4] ? mdu_res : alu_res;
// 分支条件归一化到 bit0（比较取反用 inv_flag）
assign EX_result = {exec_res[31:1], exec_res[0] ^ inv_flag_reg};

// 打拍送给 LSU
always_ff @(posedge clock)
    if (valid_last & ready_next) begin
        rd_reg <= rd;  R_wen_reg <= R_wen;  mem_wen_reg <= mem_wen;
        R_wen_reg <= R_wen; branch_pc_reg <= branch_pc;
        pred_taken_reg <= pred_taken_in; pred_target_reg <= pred_target_in;
    end
```

与后级的联系：EX_result 一方面回送 Control（分支条件/跳转目标），另一方面经 LSU 透传作为普通指令的写回值；mem_wen/mem_ren/R_wen/rd/funct3 等控制信号被逐级打拍送到 LSU 与 WBU；pred*_reg 则供 Control 在 EX 级判断“预测对不对”。

5.1 ALU（整数运算）

输入	输出	作用
d1/d2[31:0], choice[4:0]	res[31:0]	加减、逻辑、移位、比较

比较类结果统一放 res[0]。加/减通过 add 模块复用一套加法器。

关键代码（ALU.sv）

```

unique case (choice)
  `alu_add: res = add_result;           // 复用 add 模块
  `alu_sub: res = add_result;
  `alu_sll: res = d1 << d2[4:0];
  `alu_srl: res = d1 >> d2[4:0];
  `alu_sra: res = $signed(d1) >>> d2[4:0];
  `alu_signed_comparator: res[0] = ...; // 比较结果放 bit0
  `alu_unsigned_comparator: res[0] = (d1 < d2);
  `alu_equal: res[0] = (d1 != d2);
  // M 类编码在本模块只返回 0, 由 MDU 接管
endcase

```

res 返回 EXU, 与 MDU.res 二选一后成为 EX_result。

5.2 MDU (RV32M 乘法)

输入	输出	作用
d1/d2[31:0], op[4:0]	res[31:0]	mul/mulh/mulhsu/mulhu/div/divu/rem/remu

- 乘法用 64 位中间结果一次算出, 取低/高 32 位;
- 除法用恢复余数法逐位试减 (组合展开, 不跨拍);
- 边界语义: 除零 商=-1/余=被除数; INT_MIN/-1 溢出 商=INT_MIN/余=0。

Control/EXU 的配合: alu_opcode[4]=1 表示 M 扩展, EXU 选 MDU 输出, 否则选 ALU 输出 (exec_res = alu_opcode_reg[4] ? mdu_res : alu_res)。

关键代码 (MDU.sv)

```

// 乘法: 64 位中间结果, 取低/高 32 位
assign prod_ss = $signed(d1) * $signed(d2); // mulh
assign prod_uu = {32'b0, d1} * {32'b0, d2}; // mulhu

// 除法: 恢复余数法逐位试减 (组合展开)
for (int i = 31; i >= 0; i--) begin
  rem = {rem[31:0], core_a[i]};
  if (rem >= {1'b0, core_b}) begin rem = rem - {1'b0, core_b}; q_u[i] = 1'b1; end
end

// 边界: 除零商=-1/余=被除数; INT_MIN/-1 溢出商=INT_MIN/余=0

```

res 与 ALU.res 二选一后成为 EX_result (见 §5), 之后与普通整数指令走完全相同的访存、写回与前递路径——这就是把 M 扩展做在“运算单元”而非“指令通路”上的好处。

6. 访存级 LSU 与 DCache

6.1 LSU

LSU 内部有三级寄存器 (`*_reg` → `*_reg_pipe` → `*_reg2`)，同时对外暴露三个前递观察点：`MEM` (M1)、`MEM_PIPE` (M2)、`MEM2` (M3)。

输入	作用
<code>mem_ren/mem_wen</code>	load/store 使能
<code>Ex_result[31:0]</code>	地址 (或 ALU 结果)
<code>rs2_value[31:0]</code>	store 数据
<code>funct3[2:0]</code>	访问宽度 (byte/half/word)
<code>rd/R_wen/csr_wen/jump_flag/rd_value/pc</code>	透传控制与写回候选值

输出	作用
<code>addr/wen/wdata/mask</code>	发给 D-Cache/总线的访存请求
<code>rdata</code>	从 D-Cache 读回
<code>Ex_result_pipe/rd_pipe/mem_ren_pipe/...</code>	M2 前递观察点
<code>forward_val_wb/...</code>	M3 前递观察点
<code>rdata_wb_raw/funct3_wb/...</code>	交给 WBU 的字节/回读信息

地址与掩码：`addr = {..., 2'b00, addr[15:0]}` (字对齐、保留 offset)，
`mask = funct3[1:0]` (00=byte, 01=half, 10=word)。

前递值选择：跳转/CSR 指令前递 `rd_value`，普通指令前递 `Ex_result`。

它在做什么：把 EX 的结果多存两拍 (方便观察与后续再流水)，并在真正那一拍把地址/写使能/写数据/掩码发给数据总线，同时把读回的数据锁存下来。
它不判断指令类型，只负责“把带过来的东西按拍传递”。

关键代码 (`LSU.sv`)

```

// 三级移位: reg -> reg_pipe -> reg2
always_ff @(posedge clock) if (ready_next) begin
    mem_ren_reg2 <= mem_ren_reg_pipe;  Ex_result_fwd_reg2 <= Ex_result_fwd_reg_pipe;
    mem_ren_reg_pipe <= mem_ren_reg;    Ex_result_fwd_reg_pipe <= Ex_result_fwd_reg;
    mem_ren_reg <= mem_ren;              Ex_result_fwd_reg <= Ex_result;
    rd_reg2 <= rd_reg_pipe; ...          rdata_reg2 <= rdata;    // 回读数据锁存
end

// 对外总线 (字对齐 + 宽度掩码)
assign addr  = {Ex_result_addr_reg_pipe[31:18], 2'b00, Ex_result_addr_reg_pipe[15:0]};
assign mask  = funct3_reg_pipe[1:0];
assign wdata = rs2_value_reg_pipe;

```

与后级的联系: `addr/wen/wdata/mask` 交给 DCache 访问外部存储;
`rdata_wb_raw/funct3_wb/rd_wb/forward_val_wb` 交给 WBW 做最终写回;
同时 `*_pipe` (M2) 与 `*_wb` (M3) 直接作为前递源回到 Control。

6.2 DCache (数据缓存, 写直达)

输入	作用
<code>cpu_en</code>	本拍是否真的访存 (load 或 store)
<code>cpu_wen</code>	1=store, 0=load
<code>cpu_addr/wdata/mask</code>	访存请求
<code>mem_rdata[31:0]</code>	外部 DRAM 读回 (已按 mask 抽取)

输出	作用
<code>cpu_rdata[31:0]</code>	按 mask/offset 抽取后的读数据
<code>mem_addr/wen/wdata/mask</code>	转发给 DRAM
<code>hit/miss</code>	访问结果
<code>hit_cnt/miss_cnt</code>	命中/缺失计数

一致性策略: store 始终写穿到 DRAM; 命中时同步更新缓存行 (按字节掩码合并);
load 缺失回填一行 (为拿到完整 32 位字, load 时向外部发“整字读” `mask=10`, 再由缓存自己做字节/半字抽取)。

它在做什么: 在 LSU 和外部数据总线之间做一层“查表 + 转发”。命中直接返回缓存值; 缺失旁路 DRAM 并回填; 写请求原样写穿到 DRAM。它不改变访存语义。

关键代码 (DCache.sv)

```
assign hit          = cpu_en && valid[index] && (tag[index] == tag_in);
assign mem_mask     = cpu_wen ? cpu_mask : 2'b10;    // load 时整字读，便于回填整行
assign mem_addr     = cpu_addr;    assign mem_wen = cpu_wen;    // 写直达

// 命中时按 mask/offset 从缓存字抽取读数据；缺失直接旁路 mem_rdata
assign cpu_rdata = hit ? <按 mask/offset 抽取 data[index]> : <按 mask 抽取 mem_rdata>;
```

与后级的联系：读回的 `cpu_rdata` 进入 `LSU.reg2`，最终由 `WBU` 写回寄存器；
写请求被原样转发到 DRAM（写直达），保证与外部存储一致。
它位于 `LSU` 与“内存/外设”之间，是唯一接触外部数据总线的模块。

7. 写回级 WBU

输入	作用
<code>MEM_Rdata_in</code>	load 原始读回
<code>funct3_in</code>	load 宽度（符号/零扩展）
<code>Ex_result_in</code>	ALU/MDU 结果
<code>rd_value_in</code>	跳转/CSR 预置值
<code>csr_wen_in/R_wen_in/mem_ren_in/jump_flag_in</code>	控制标志
<code>pc_in</code>	本条指令 PC

输出	作用
<code>rd_value_next[31:0]</code>	最终写回值（三选一）
<code>csrd[31:0]</code>	CSR 写数据
<code>csr_wen_next/R_wen_next</code>	写使能
<code>rd_next/pc_out</code>	目标寄存器号/PC
<code>valid_next</code>	提交有效

最终选择优先级：

```
jump/CSR -> rd_value_in
load      -> 按 funct3 扩展后的 rdata
其它      -> Ex_result_in
```

`WBU` 是整条数据通路唯一的“提交点”，`debug_wb_*` 也接在这里。

它在做什么：把三条候选写回路径（ALU/MDU 结果、load 读回、跳转/CSR 预置值）归一到唯一一个 `rd_value_next`，并产生最终的最宽 load 扩展结果。

关键代码（`WBU.sv`）

```
// 按 funct3 把读回数据做符号/零扩展
case (funct3_reg)
  3'b000: rdata_wb = rdata_8i;    // lb
  3'b001: rdata_wb = rdata_16i;   // lh
  3'b010: rdata_wb = MEM_Rdata_reg;
  3'b100: rdata_wb = rdata_8u;    // lbu
  3'b101: rdata_wb = rdata_16u;   // lhu
endcase

// 最终写回三选一
assign wb_sel_jmp_csr = jump_flag_reg | (!csr_wen_reg);
assign rd_value_next  = wb_sel_jmp_csr ? rd_value_reg
                                : (mem_ren_reg ? rdata_wb : Ex_result_reg);
assign R_wen_next     = R_wen_reg & valid_reg;
```

与后级的联系：rd_value_next 回到 Reg_Stack.RegisterFile 写通用寄存器，
csrd/csr_wen_next 回到 CSR 写系统寄存器，rd_next/R_wen_next 同时是
Data_hazard 的 WB 前递源；debug_wb_* (=WBU_valid/pc/R_wen/rd/rd_value)
是差分测试的观测点。至此“取指→译码→执行→访存→写回”形成闭环。

8. 冒险处理：Data_hazard 与 Control

8.1 Data_hazard （前递仲裁器）

只产生选择编码，不搬运数据；真正搬运在 Control。

输出编码	数据源
3'b001	EXU （EX 级结果）
3'b010	MEM （M1 级结果）
3'b101	MEM_PIPE （M2 级）
3'b011	MEM2 （M3 级）
3'b100	WB （写回值）
3'b000	不前递 （用寄存器堆读值）

优先级从近到远：EXU > MEM > MEM_PIPE > MEM2 > WB。
若 MEM 级是 load，则数据尚未可用，返回 000，交由 Control 产生 load-use 暂停。

关键代码（Data_hazard.sv）

```

assign exu_hit  = EXU_R_Wen  && (EXU_rd ==IDU_rs1) && (EXU_rd !=0);
assign mem_hit  = MEM_R_Wen  && (MEM_rd ==IDU_rs1) && (MEM_rd !=0);
assign pipe_hit = MEM_PIPE_R_Wen && MEM_PIPE_valid && (MEM_PIPE_rd==IDU_rs1) && ...;
assign mem2_hit = MEM2_R_Wen && MEM2_valid && ...;
assign wb_hit   = WB_R_Wen   && (WB_rd  ==IDU_rs1) && ...;

assign IDU_rs1_choice = exu_hit  ? 3'b001 :
                        mem_hit  ? (MEM_mem_ren ? 3'b000 : 3'b010) : // load 未好 -> 不选
                        pipe_hit ? 3'b101 :
                        mem2_hit ? 3'b011 :
                        wb_hit   ? 3'b100 : 3'b000;

```

它在做什么：只做“比较寄存器号 + 给优先级编码”，不搬运数据。

Control 拿到编码后再从 5 个源里真正挑值（见 §8.2），两者合起来才是完整前递。

8.2 Control（全局协同核心）

它在做什么：它是整机的“交通指挥”。Data_hazard 只说“谁最新”，Control

才真正把最新值搬回去；同时它统一决定“暂停、冲刷、重定向、接收中断”，

并把 DNPC 交给 IFU。可以把它理解为前递网络 + 冒险控制器 + 重定向控制器的合体。

Control 做四件事：

(1) 前递多路选择

```

EXU_rs1_in = choice==EXU ? Ex_result :
              choice==MEM ? MEM_Ex_result :
              choice==MEM_PIPE ? MEM_PIPE_Ex_result :
              choice==MEM2 ? MEM2_Ex_result :
              choice==WB ? WB_rd_value : IDU_rs1_value;

```

rs2 同理。这样译码级拿到的就是“最新可用值”。

(2) load-use 暂停

```

exu_load_use  = EXU_mem_ren  & (EXU_rd==rs1|rs2) & rd!=0
mem_load_use  = MEM_mem_ren  & ...
pipe_load_use = MEM_PIPE_mem_ren & ...
IFU_stall     = exu | mem | pipe

```

命中时冻结 IFU/IDU 并清空执行级，插入气泡。

(3) 分支预测校验与重定向

```

actual_taken  = branch_taken | jump_flag
actual_target = jump_flag ? Ex_result : branch_pc
bp_mispredict = (actual_taken != EXU_pred_taken)
               | (actual_taken & (actual_target != EXU_pred_target))
dnpc          = bp_mispredict ? (actual_taken ? actual_target : EXU_pc+4) : ...

```

预测正确时不冲刷；预测错误时冲刷前端并重定向，同时更新预测器。

(4) 中断精确接收

```
intr_safe = IDU_valid
            & ~(branch_flag|jump_flag|fence_i_flag)    // 执行级没有待定重定向
            & ~mret_flag & ~ecall_flag                // 当前指令不是系统指令
            & ~IFU_stall;
intr_take = intr_pending & intr_safe;
```

只在流水线“干净”时接收中断，保证 mepc 指向真正被中断的指令。

输出汇总

输出	作用
EXU_rs1_in/rs2_in	前递后的操作数
IFU_stall	前端暂停
EXU_inst_clear	清空进入执行级的指令（插泡/冲刷）
dnpc/dnpc_flag	下一 PC 及重定向请求
intr_take	本拍接收中断
bp_update_en/taken/target	预测器更新
bp_mispredict	预测错误标志

与前后级的联系：向前，它把 EXU_rs1_in/rs2_in 送回 IDU（前递），把 dnpc/dnpc_flag 送回 IFU（重定向），把 IFU_stall 同时送给 IFU/IDU（暂停）；向后，它把 bp_update_* 回灌给 Branch_Predictor，把 intr_take 送给 IDU-CSR（陷入），把 EXU_inst_clear 送给 EXU（插泡/冲刷）。也就是说，几乎所有跨模块的“反向控制”都经过 Control。

9. 典型指令走查

9.1 add x3, x1, x2

IF 取指 → ID 译码出 alu_opcode=alu_add、add1=rs1、add2=rs2 → EX 中 ALU 算出 EX_result → LSU 三级透传 → WBU 选 Ex_result 写回 x3。

9.2 lw x5, 0(x6)

ID 给出 mem_ren=1、add1=rs1、add2=imm；EX 算地址；M2 向 DCache 发整字读；M3 回读并按 funct3 扩展；WBU 选 rdata 写回 x5。若 x5 立刻被下一条指令使用，Data_hazard 发现 MEM 级是 load，Control 暂停一拍，待结果可用后前递。

9.3 sw x7, 4(x6)

ID 给出 `mem_wen=1`、`add1=rs1`、`add2=imm`、`rs2_value=x7`；EX 算地址；
M2 向 DCache 发写直达请求（`wen=1`、`mask=funct3[1:0]`、`wdata=x7`），
同时若缓存命中则按掩码合并缓存行。无写回（`R_wen=0`）。

9.4 mul x9, x7, x8 / div x9, x7, x8

ID 识别 `funct7=00000001`，给出 `alu_opcode` 最高位为 1 的 M 编码；
EX 中 MDU 与 ALU 并行计算，`exec_res` 选择 MDU 结果；
后续与普通 ALU 指令完全相同（写回 `x9`）。

9.5 beq x1, x2, label

IF 时 `Branch_Predictor` 查询该 PC：若 BTB 命中且 BHT 预测 taken，则 PC 直接跳到预测目标（投机执行）。指令到 EX 后：

- 预测正确 → 不冲刷，继续；
- 预测错误 → `Control` 发出 `dnpc=实际目标 or pc+4`，冲刷前端并更新预测器。
分支条件由 EXU 归一化到 `EX_result[0]`（`inv_flag` 处理取反）。

9.6 jal x1, label / jalr x1, 0(x5)

无条件跳转，目标在 EX 计算（`pc+imm` 或 `rs1+imm`）。预测器为它们维护 BTB 项，首次会 miss 并纠正，之后可正确预测。

9.7 ecall

ID 检测 `inst==0x00000073`：CSR 写 `mepc<=pc`、`mcause<=11`、`MIE<=0`；
`Control` 令 `dnpc=mtvec`，前端冲刷。测试平台用它作为“程序结束”信号。

9.8 中断

CSR 置 `intr_pending`（如软件写 `mip.MSIP` 或定时器 `mtime>=mtimecmp`）；
`Control` 在流水线安全时 `intr_take=1`：CSR 记录 `mepc/mcause`、关全局中断，
`dnpc=mtvec` 冲刷前端。handler 执行后 `mret`：`MIE<=MPIE`、`PC<=mepc` 返回。

10. 验证方法

差分测试平台位于 `cdp-tests/`，逐条比对 RTL 与 golden model 的写回。

```
cd cdp-tests

# 单条 (RV32I / RV32M)
make clean
make run TEST=addi CPU_DIR=../v9
make run TEST=mul CPU_DIR=../v9
make run TEST=divu CPU_DIR=../v9
```

```
# 全量回归（47 条，含 RV32IM；自动跳过中断用例）
CPU_DIR=../v9 python3 run_all_tests.py

# 中断独立验证（软件中断 / 定时器中断）
python3 tools/gen_intr_test.py
python3 tools/gen_timer_test.py
INTR_TEST=1 make run TEST=intr CPU_DIR=../v9
INTR_TEST=1 make run TEST=intr_timer CPU_DIR=../v9
```

新增测试的生成工具在 `cdp-tests/tools/`：

- `rv_asm.py`：最小 RV32I/RV32M 汇编器；
- `gen_m_tests.py`：生成 8 条 M 扩展定向测试；
- `gen_intr_test.py` / `gen_timer_test.py`：生成中断测试。

11. 如何继续扩展

1. **Cache**：把 ICache/DCache 的 miss 接到 IFU.stall / LSU 暂停，即可支持多周期存储器；把写策略换成 write-back + dirty 位即可支持写回。
2. **分支预测**：把 BHT 换成 gshare/局部历史，或在 BTB 中加入 is_call/is_ret 与 RAS。
3. **中断**：CSR 中 meip 已预留，可接入外部中断控制器；mtvec 可扩展向量模式。
4. **乘除法**：把 MDU 改为多周期流水单元并在 Control 增加 mdu_busy 暂停，可显著提高时序频率。
5. **特权**：可增加 U 模式与 mstatus.MPP、mret 特权切换。

12. 一页速查：模块 → 作用

模块	一句话作用
IFU	维护 PC，在顺序/暂停/预测/重定向之间选择下一 PC
ICache	取指缓存，命中加速、缺失旁路并回填
Branch_Predictor	BHT+BTB 动态预测跳转方向与目标
IDU	译码：控制信号、操作数、立即数、CSR/系统标志、预测信息透传
Reg_Stack	通用寄存器 + CSR 的统一读入口
CSR	机器模式寄存器与中断判决、陷入/返回状态机
EXU	ALU/MDU 运算、分支条件归一化、跳转目标
ALU / MDU	整数运算 / 乘除法
LSU	访存三级流水 + 多级前递观察点
DCache	数据缓存（写直达、写不分配）

模块	一句话作用
WBU	最终写回值选择与提交
Data_hazard	前递仲裁：谁的值最新
Control	前递选择、暂停、冲刷、重定向、中断接收